

This are the 10 question topics multiple choice: 1. use case diagram 2. class diagram 3. Sequence diagram/Activity 4. Solid 5. C4 6. DDD 7. GoFDP 8. Agile Requirements (User stories) 9. Data Bases 10. Micro Services

1. Use Case Diagram

What it is

- A UML diagram that models **functional behavior** of a system from the **user's perspective**.
- Shows how **actors** interact with **use cases** inside a **system boundary**.

Core elements

- **Actors:** external entities (people or systems).
- **Use cases:** system functionalities (ovals).
- **System boundary:** separates system from environment.
- **Relationships:**
 - *Include:* mandatory reuse of behavior.
 - *Extend:* optional/conditional behavior.
 - *Generalization:* inheritance between actors or use cases.
 - *Association:* communication link actor ↔ use case.

Purpose

- Define system scope.
- Communicate requirements clearly to stakeholders.
- Basis for detailed use case specifications.

Source: SE_03.pdf, pp. 25–31

Model-based requirements specification

25/39

Use case diagrams

Use Cases

Use cases were first proposed in [Jacobson et al. 1992] as a model-based technique to **document the functionalities** of a planned or existing system.

The use case approach is based on two concepts that are used in conjunction with one another:

- ▶ Use Case Diagram
- ▶ Use case specifications

Use Case Diagram

Use Case Diagrams (UCDs) describe the **functional behavior of a system** from the user's perspective. They show how **actors** interact with **use cases** within system boundaries.

26/39

Use-case diagram elements

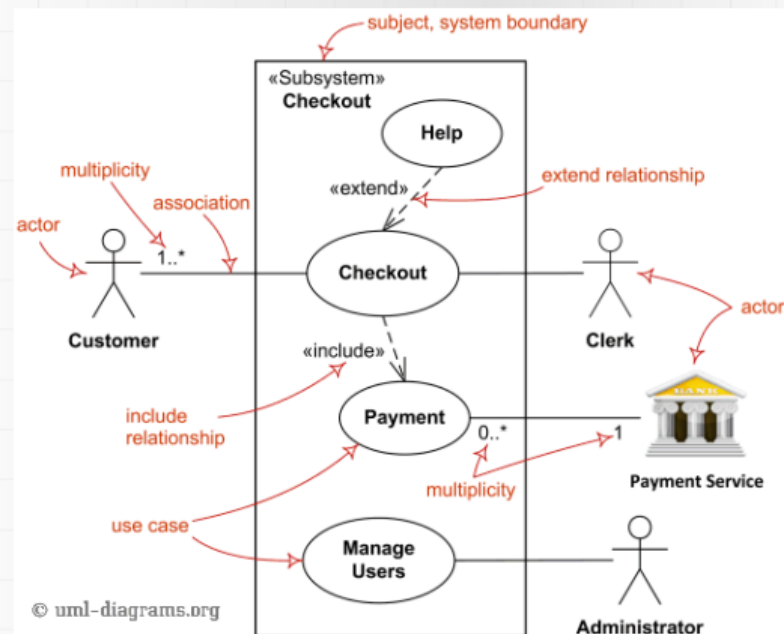


Figure: Use case diagram elements⁴

27/39

⁴<https://uml-diagrams.com>

Use Case Modeling (UML)

Key categories

1. **Use cases** – shown as *ovals*, each naming a system function. The name may also be placed below the oval.
2. **Actors** – external entities (people or systems) interacting with the system. Depicted as stick figures (or icons, or rectangles stereotyped with «actor»).
3. **System boundary** – rectangle enclosing all use cases; separates internal and external parts.

Relations between elements

1. **Include** – one use case *includes* another to reuse its behavior. Example: "Navigate to destination" *includes* "Input destination".
2. **Extend** – one use case *extends* another conditionally at a defined *extension point*. Example: "Download traffic info" *extends* "Navigate to destination" if "Avoid congestion".
3. **Generalization** – defines inheritance between actors or use cases. Example: "Service mechanic" and "Customer support" → "Employee".
4. **Communication links** – associations between actors and use cases where interaction occurs.

28/39

Use-case diagram example

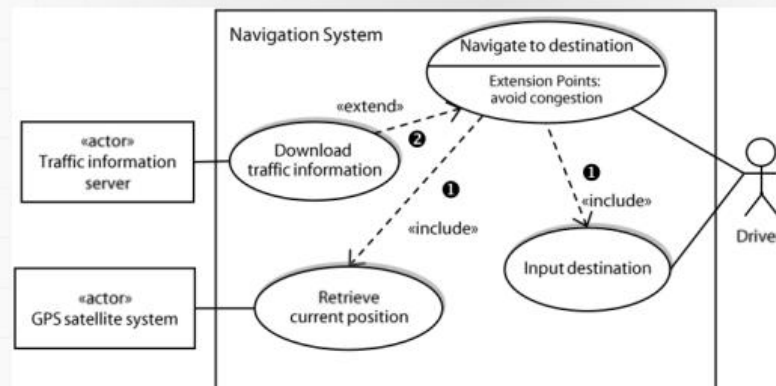


Figure: Use case diagram example⁵

⁵ (Pohl 2010)

Use case specifications

Use Case Diagrams show only **relations and scope**, not internal behavior. To document the detailed interaction between an actor and the system, each use case must be described

- ▶ **textually using a specification template**
- ▶ optionally **complemented by behavioral models**: activity (workflow), sequence (message exchange), state machine (lifecycle).

Use-case specification example

Section	Content
Designation	UC-12-37
Name	Navigate to destination
Authors	John Smith, Sandra Miller
Priority	Importance for system success: high Technological risk: high
Criticality	High
Source	C. Warner (domain expert for navigation systems)
Responsible	J. Smith
Description	The driver of the vehicle types the name of the destination. The navigation system guides the driver to the desired destination.
Trigger event	The driver wishes to navigate to his destination.
Actors	Driver, traffic information server, GPS satellite system
Pre-condition	The navigation system is activated.
Post-condition	The driver has reached his destination.
Result	Route guidance to the destination
Main scenario	1. The navigation system asks for the desired destination. 2. The driver enters the desired destination. 3. The navigation system pinpoints the destination in its maps. 4. On the basis of the current position and the desired destination, the navigation system calculates a suitable route. 5. The navigation system compiles a list of waypoints. 6. The navigation system shows a map of the current position and shows the route to the next waypoint. 7. When the last waypoint is reached, the navigation system shows "destination reached" on the screen.
Alternative scenario	4a. Calculation of the route must honor traffic information and avoid traffic congestions. 4a1. The navigation system queries the server for updated traffic information. 4a2. The navigation system calculates a route that does not contain any traffic congestions.
Exception scenarios	Trigger event: The navigation system does not receive a GPS signal from the GPS satellite system.
Qualities	→ QR.04 (reaction time upon user input) → QR.15 (operating comfort) (QR = quality requirements)

2. Class Diagram

Purpose

- UML structural diagram showing the **static structure** of a system.
- Models **classes, attributes, operations, and relationships**.

Core elements

- **Class:** name, attributes, operations.
- **Classifier types:** class, interface, enum, data type.
- **Visibility:** + public, - private, # protected.
- **Stereotypes:** «entity», «control», etc.

Relationships

- **Association:** structural link (with roles, navigability, multiplicity).
- **Aggregation:** weak whole–part (empty diamond).
- **Composition:** strong ownership, lifecycle dependency (filled diamond).
- **Generalization:** inheritance (triangle arrow).
- **Dependency:** usage relationship (dashed arrow).

Multiplicity

- Examples: 1, 0..1, *, 1..*, 2..5
- * = unspecified upper bound (not mathematical infinity).

Uses

- Domain modeling
- OOP code derivation
- Information and subsystem design

Source: SE_06_07.pdf, pp. 17–21

UML Structure Diagrams

Overview

Diagram

A UML diagram is a partial **graphical representation** (view) of a **model** of a system under design, implementation, or already in existence. UML diagram contains graphical elements (symbols) - UML nodes connected with edges (also known as paths or flows) - that represent elements in the UML model of the designed system.

Structural diagrams:

- ▶ Show the **static structure** of a system and its parts.
- ▶ Represent concepts at different abstraction and implementation levels.
- ▶ Do **not** describe time-related or dynamic behavior.
- ▶ May show relationships to behaviors of the classifiers.

Core Structure Diagrams

- ▶ **Class Diagram** – structure of designed system, subsystem or component as related classes and interfaces, with features, constraints and relationships.
Elements: class, interface, feature, association, generalization, dependency.
- ▶ **Object Diagram** – instance-level class diagram; shows concrete objects and links (snapshot of system state).
Elements: instance specification, slot, link.
- ▶ **Package Diagram** – organizes elements into packages and shows their dependencies.
Elements: package, dependency, import, merge.
- ▶ **Composite structure diagram** – internal structure of a classifier
Elements: classifiers, roles, parts, connectors.
- ▶ **Component Diagram** – components and dependencies between them
Elements: component, interface, port, connector, artifact, realization.
- ▶ **Deployment diagram** – Shows physical architecture of the system as deployment (distribution) of software artifacts to deployment targets.
Elements: devices, execution environments, nodes, artifacts, manifestations, components

Class Diagram

Basic info

- ▶ Shows the **logical structure** of the system: classes, their attributes, operations, and relationships.
- ▶ Each class is a **classifier**, i.e. a template for objects that share structure and behavior.
- ▶ Often used to:
 - ▶ derive OOP code structure,
 - ▶ analyze domain models (conceptual view),
 - ▶ analyze information needs,
 - ▶ visualize architecture of subsystems.
- ▶ Level of abstraction: between conceptual domain model and implementation-level code.

18/36

Class Diagram

Classifiers

- ▶ **Classifier** – an abstract element defining a set of instances sharing the same features.
 - ▶ Examples: class, interface, enumeration, data type, component.

- ▶ **Class notation:**

ClassName
attribute_1 : Type
operation(parameter : Type) : ReturnType

- ▶ Classes may have:
 - ▶ *visibility* (+ public, – private, # protected),
 - ▶ *stereotypes* («entity», «control», etc.),

19/36

Class Diagram

Relationships

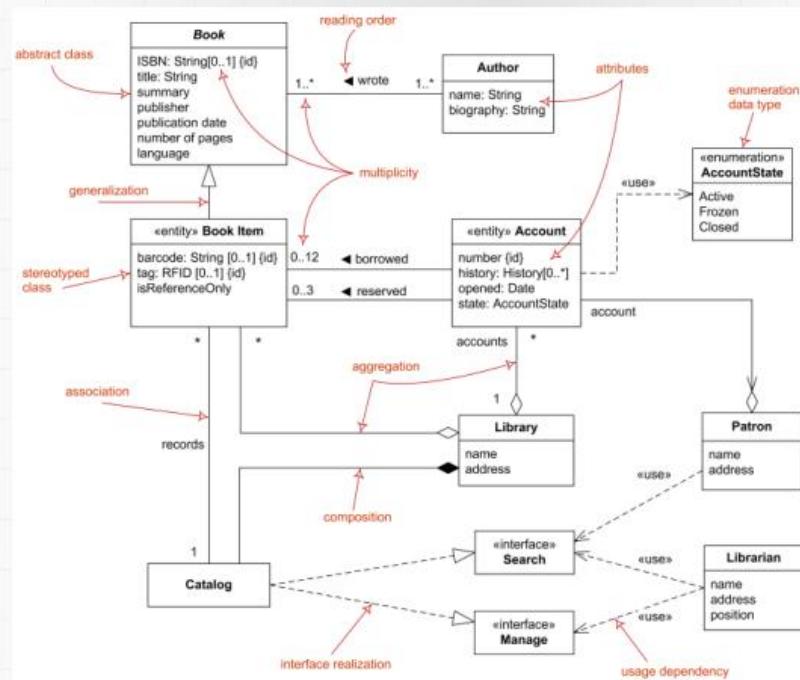
- ▶ **Association** – structural connection between classifiers (e.g. Student–Course).
 - ▶ may be named, navigable, have roles and multiplicities (e.g. 1..*, 0..1).
- ▶ **Aggregation** (empty diamond) – “whole-part” relationship with weak ownership.
- ▶ **Composition** (filled diamond) – strong ownership; parts are destroyed with the whole (*existence dependency*).
- ▶ **Generalization** (triangle arrow) – inheritance relation; subclass specializes superclass.
- ▶ **Dependency** (dashed arrow) – one element uses or depends on another.

Class Diagram

Multiplicity and Association Details

- ▶ **Multiplicity** – number of instances related to another class.
 - ▶ $\infty \neq *$
 - ▶ asterisk means that the upper bound is not specified (unlimited in the sense of the model, not infinite in mathematics)
 - ▶ Examples: 1, 0..1, *, 1..*, 2..5
- ▶ **Arity** – number of association ends (typically 2).
- ▶ **Role names** – clarify semantic meaning of association ends.
- ▶ **Association class** – allows adding attributes or operations to a relationship.
- ▶ Enables modeling of real-world constraints (e.g. “each student may be enrolled in many courses, each enrollment has a grade”).

Class diagram



Source: uml-diagrams.org, great UML 2.5 standard compendium

3. Sequence Diagram / Activity Diagram

Activity Diagram

What it models

- **Workflows and processes.**
- Emphasizes **control flow** and **data flow**.

Key concepts

- **Actions, object nodes, control nodes.**
- **Token flow semantics** (control tokens vs object tokens).
- **Decision / Merge, Fork / Join** for branching and parallelism.
- **Swimlanes** (activity partitions) for responsibility separation.

Best for

- Business processes
- Data-driven workflows
- Parallel execution logic

Source: SE_08.pdf, pp. 9–19

Sequence Diagram

What it models

- **Interactions between lifelines over time.**
- Emphasizes **temporal ordering** of messages.

Key elements

- **Lifelines** (objects, actors, components).
- **Messages:**
 - Synchronous (blocking)
 - Asynchronous (non-blocking)
 - Return
 - Create / Destroy
- **Activation bars** (execution specifications).

- **Combined fragments:** alt, opt, loop, par.

Best for

- Use case realizations
- Interaction protocols
- Scenario modeling

Source: SE_08.pdf, pp. 20–31

4. SOLID

What it models

- Principles for good object-oriented design.
- Focus on maintainability, extensibility, and low coupling.

Key principles

- **Single Responsibility (SRP):** one class → one reason to change.
- **Open/Closed (OCP):** open for extension, closed for modification.
- **Liskov Substitution (LSP):** subtypes must replace base types safely.
- **Interface Segregation (ISP):** small, client-specific interfaces.
- **Dependency Inversion (DIP):** depend on abstractions, not concretes.

Best for

- Avoiding god objects and tight coupling
- Improving testability and changeability
- Structuring maintainable OOP systems

Source: SE_12.pdf, pp. 4–5



Motivation

Why pure Object-Oriented is not enough

Typical symptoms of naive OOP projects:

- ▶ **Tight coupling** between classes.
- ▶ So-called "**God objects**" knowing and doing too much.
- ▶ **Mixed concerns:** Business logic tangled with infrastructure (user interface, database, communication).

As a result we end up with **code that works, but resists change.**

- ▶ How to deal with that?

4/39

Foundations

SOLID Principles

Before discussing patterns, we must understand the principles of good object-oriented design.

- ▶ **S – Single Responsibility Principle:** A class should have one, and only one, reason to change. One task, one purpose, etc.
- ▶ **O – Open/Closed Principle:** Entities should be open for extension, but closed for modification.
- ▶ **L – Liskov Substitution Principle:** Subtypes must be substitutable for their base types.
- ▶ **I – Interface Segregation Principle:** Many client-specific interfaces are better than one general-purpose interface. Client should not be forced to implement unnecessary methods that they don't use.
- ▶ **D – Dependency Inversion Principle:** Depend upon abstractions, not concrete concepts.

5. C4 Model

What it is

- A lightweight architecture modeling approach by **Simon Brown**.
- Focused on **clarity and different abstraction levels**, especially for distributed systems.

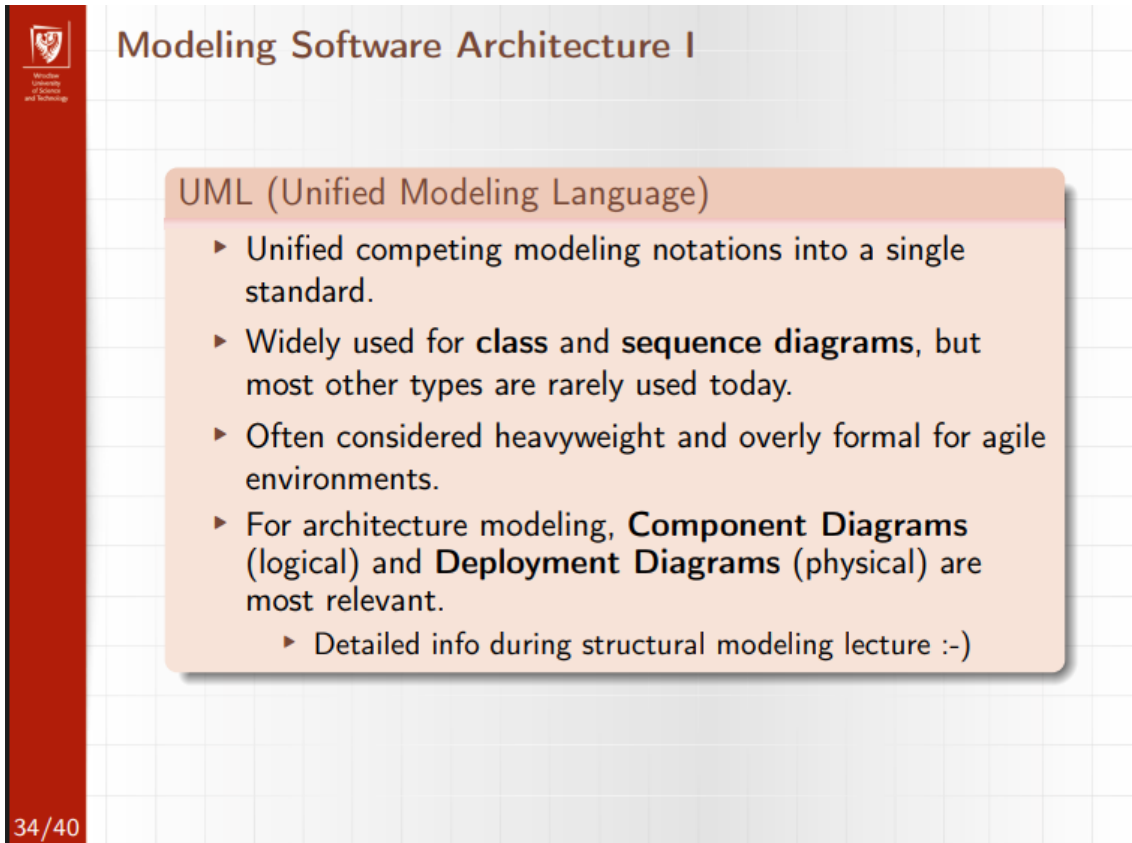
Four levels

1. **Context** – system + external actors.
2. **Container** – applications, services, databases.
3. **Component** – internal building blocks of a container.
4. **Code (Class)** – optional, detailed implementation view.

Key characteristics

- Hierarchical abstraction.
- Tool-friendly, less formal than UML.
- Emphasizes communication over completeness.

Source: SE_05.pdf, pp. 34–38



The slide features a red vertical bar on the left with a university logo. The main title is 'Modeling Software Architecture I'. A light orange box contains the text 'UML (Unified Modeling Language)' followed by a bulleted list of four points. The bottom left corner shows the page number '34/40'.

Modeling Software Architecture I

UML (Unified Modeling Language)

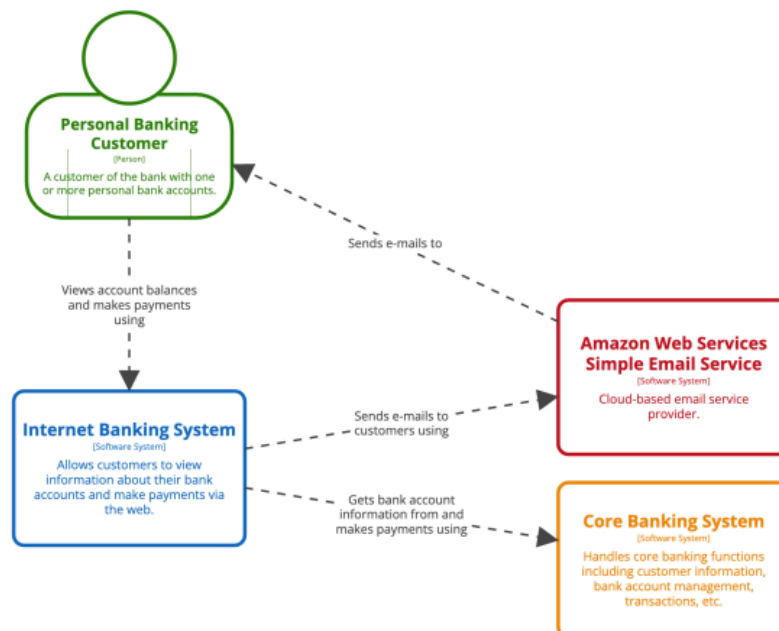
- ▶ Unified competing modeling notations into a single standard.
- ▶ Widely used for **class** and **sequence diagrams**, but most other types are rarely used today.
- ▶ Often considered heavyweight and overly formal for agile environments.
- ▶ For architecture modeling, **Component Diagrams** (logical) and **Deployment Diagrams** (physical) are most relevant.
 - ▶ Detailed info during structural modeling lecture :-)

34/40

C4 Model

- ▶ Introduced by Simon Brown (2006–2011) as a modern alternative to UML.
- ▶ Useful for distributed architectures.
- ▶ Defines four hierarchical levels of abstraction:
 - ▶ **Context** – system and its external actors.
 - ▶ **Container** – applications, databases, and services within the system.
 - ▶ **Component** – internal building blocks of each container.
 - ▶ **Class (Code)** – detailed code-level structures (optional).
- ▶ Lightweight, tool-friendly, and widely adopted in modern software architecture.

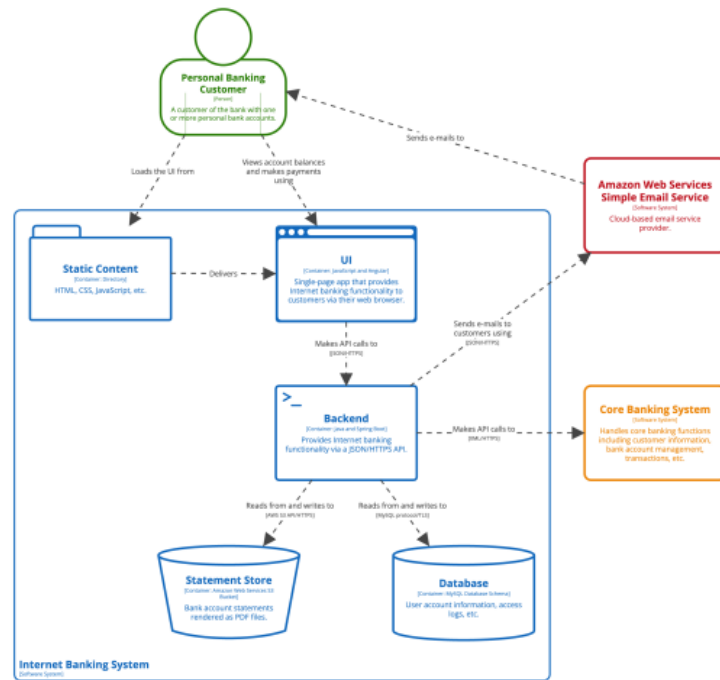
C4 Model Context



System Context View: Internet Banking System

The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

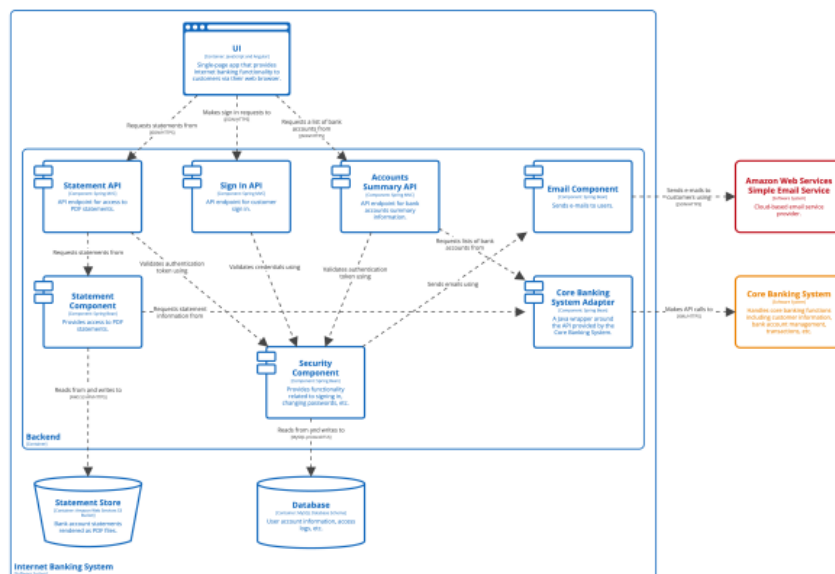
C4 Model Containers



Container View: Internet Banking System
The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

37/40

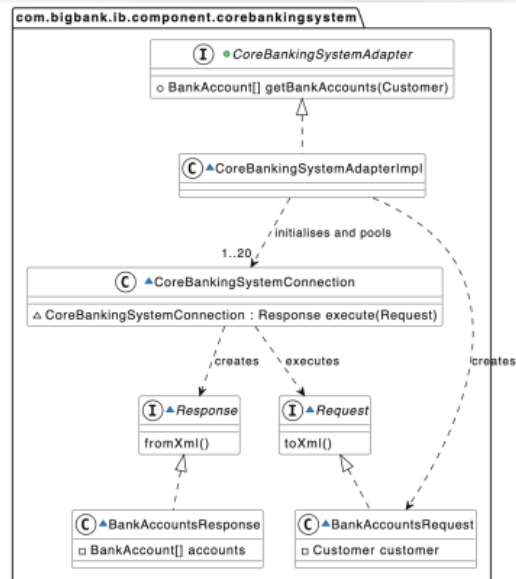
C4 Model Components



Component View: Internet Banking System - Backend
The component diagram for the Internet Banking System (Backend) | Simon Brown | c4model.com | License: CC BY 4.0

38/40

C4 Model Code



Code View: Internet Banking System - Backend - Core Banking System Adapter

A summary of the implementation details for the Core Banking System Adapter component | Simon Brown | c4model.com | License: CC BY 4.0

6. DDD (Domain-Driven Design)

What it models

- The core business domain explicitly in code.
- Separation of domain logic from technical concerns.

Key concepts

- **Ubiquitous Language:** shared vocabulary with domain experts.
- **Entity:** identity-based object.
- **Value Object:** immutable, attribute-based.
- **Aggregate:** consistency boundary.
- **Aggregate Root:** single access point to an aggregate.
- **Bounded Context:** explicit boundary where a model applies.

Levels

- **Strategic DDD:** system structure, bounded contexts.
- **Tactical DDD:** entities, aggregates, invariants.

Best for

- Complex business domains
- Large systems with multiple models
- Preventing mixed or inconsistent domain logic

Source: SE_12.pdf, pp. 27–31



University of Kent
School of Engineering and Technology

Domain-Driven Design (DDD)

Core Concepts

Domain-Driven Design (DDD)

Domain-Driven Design is an approach to software development that focuses on modeling the **core business domain** and expressing it explicitly in the code.

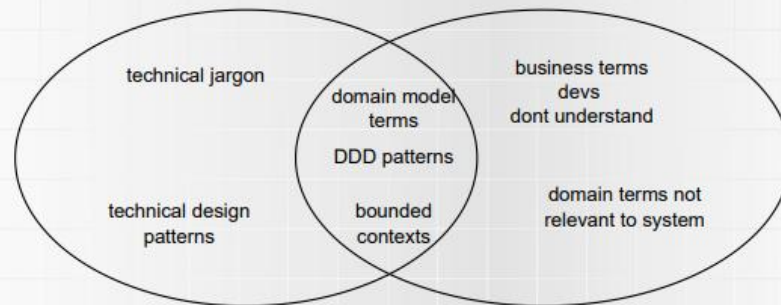
DDD emphasizes:

- ▶ a **rich domain model** that reflects business concepts,
- ▶ a **shared language** between developers and domain experts (Ubiquitous Language),
- ▶ clear **boundaries** separating the domain from technical concerns.



Domain-Driven
DESIGN
Tackling Complexity in the Heart of Software
Eric Evans
Foreword by Martin Fowler

27/39



Ubiquitous Language

Figure: Ubiquitous Language

28/39

DDD Building Blocks

- ▶ **Entity:** Defined by identity, not attributes (e.g., User with ID).
- ▶ **Value Object:** Defined by attributes, immutable, no identity (e.g., Money, Address).
- ▶ **Aggregate:** A consistency boundary that groups entities and value objects and enforces domain invariants.
- ▶ **Aggregate Root:** The single entry point to an aggregate; all external access goes through the root.
- ▶ **Bounded Context:** Explicit boundary within which a domain model is defined and applicable.

Within a bounded context:

- ▶ terms have a **precise and shared meaning**,
- ▶ the model is **logically unified**,
- ▶ code and data follow the same conceptual rules.

Outside the boundary, **different models may apply** and **explicit translation** is required.

29/39

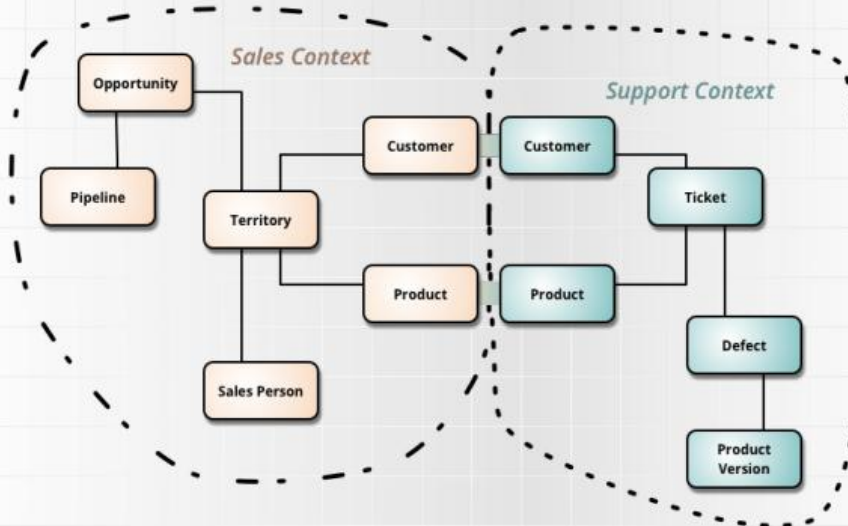


Figure: Source: M. Fowler

- Notice **Product** and **Customer** – appear in both contexts because they represent different domain concepts with different meanings, even though they share the same name.

30/39

Domain-Driven Design

Strategic vs Tactical

Aspect	Strategic DDD	Tactical DDD
Primary focus	System structure and boundaries	Domain model and code structure
Main question	<i>Where does a model apply?</i>	<i>How is the model implemented?</i>
Key concepts	Bounded Context, Context Map	Entity, Value Object, Aggregate , Repository
Level	Architecture, teams, integration	Classes, methods, invariants
Typical decisions	Context boundaries, integration style	Encapsulation, consistency rules
Failure mode	Conceptual chaos, mixed models	Anemic domain model

31/39

7. GoFDP (Gang of Four Design Patterns)

What it models

- Reusable solutions to recurring design problems.
- Structure and interaction of classes and objects.

Key characteristics

- Context-specific, not copy-paste code.
- Shared vocabulary for design decisions.
- Improves local design quality.

Classification

- **Creational:** object creation (Singleton, Factory).
- **Structural:** composition and structure (Adapter, Decorator).
- **Behavioral:** communication and responsibility (Observer, Strategy).

Limits

- Solve design-level problems, not architecture.
- Do not define system-wide structure.

Best for

- Reducing duplication
- Improving extensibility
- Communicating design intent clearly

Source: SE_12.pdf, pp. 6–9, 23–24

Design Patterns

What are they?

Design Pattern: Definition

A **design pattern** is a **proven and reusable solution** to a **recurring software design problem** that:

- ▶ applies in a **specific design context**
 - ▶ describes a **general solution**, not a concrete implementation
 - ▶ captures **best practices** derived from experience
 - ▶ provides a **shared vocabulary** for design communication
 - ▶ focuses on **structure and interaction** of software components
-
- ▶ Not ready-made solutions (copy-paste code).
 - ▶ Not a library of classes.
 - ▶ **A shared language** for communicating design decisions.
 - ▶ Proven solutions to common problems in a specific context.

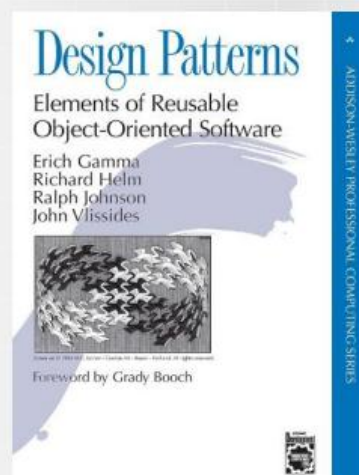
7/39

Famous Gang of Four (Gamma et al.)

- ▶ What is this famous GoF (Gamma, Helm, Johnson, Vlissides)?

Erich Gamma et al. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, Massachusetts: Addison-Wesley.

ISBN: 0-201-63361-2



8/39

Design Patterns

Classification (GoF)

- ▶ **Creational**
 - ▶ Control object creation
 - ▶ Examples: Singleton, Factory
- ▶ **Structural**
 - ▶ Organize class/object composition
 - ▶ Examples: Adapter, Decorator
- ▶ **Behavioral**
 - ▶ Assign responsibilities and communication
 - ▶ Examples: Observer, Strategy

Limits of Design Patterns

- ▶ GoF OOP design patterns improve quality **locally**, but they do not solve architectural problems.

Patterns work at the level of classes and objects. They do not address:

- ▶ Overall system structure.
- ▶ Layer dependencies.
- ▶ Module responsibilities.

Design vs Architecture

Design Pattern	Architectural Pattern
Local problem	Global decision
Classes / Objects	Modules / Layers / Components
Implementation detail	High-level structure
e.g., Strategy, Factory	e.g., MVC, Microservices, Hexagonal

8. Agile Requirements (User Stories)

User stories

- Agile replacement for detailed SRS requirements.
- Format:
As a <role>, I can <action> so that <business value>.

Key properties

- Focus on **user value**, not technical design.
- Small, testable, and negotiable.
- Stored and prioritized in the **Product Backlog**.

Related concepts

- **Tasks**: technical work items derived from a story.
- **Spikes**: time-boxed research stories to reduce uncertainty.
- **Acceptance criteria**: define when a story is “done”.

Purpose

- Support iterative development.
- Enable change and continuous feedback.

Source: SE_04.pdf, pp. 18–24

User Stories

User Stories: the Agile form of Requirements

- ▶ User stories often replace or complement detailed SRS-style traditional **software requirements** in agile environments.
- ▶ They are **brief statements of intent** that describe what the system should do for a specific user.
- ▶ Typical format:
As a <user role>, I can <activity> so that <business value>.
Example: As a registered user, I can reset my password so that I can regain access to my account if I forget it.
- ▶ Encourage focus on user value and business outcomes, not on technical details.

The Backlog

- ▶ Contains all **user stories**, plus defects, refactors, and infrastructure work.
- ▶ Maintained and prioritized by the **Product Owner**.
- ▶ Represents the team's **pipeline of value delivery**.
- ▶ Activities: identifying, prioritizing, elaborating, implementing, testing, and accepting stories.

The Backlog Example

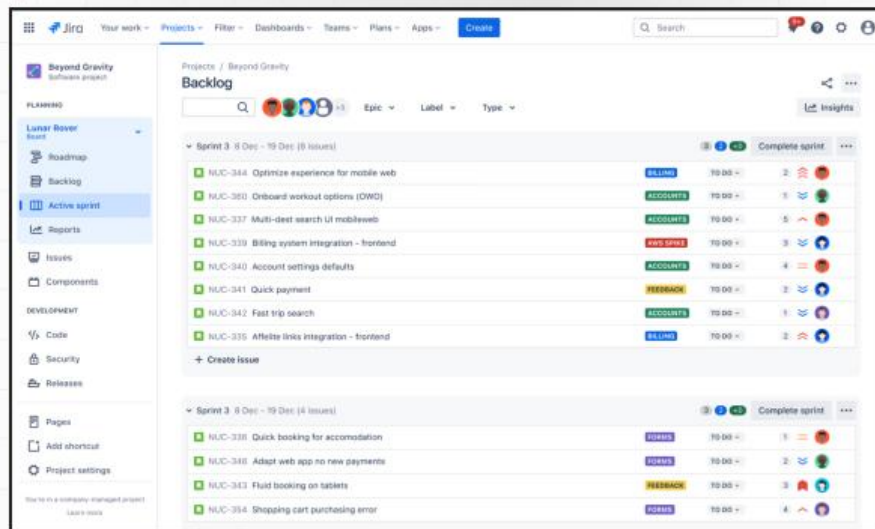


Figure: The Agile Team Backlog

Source: <https://productschool.com/blog/product-fundamentals/product-backlog-examples>

20/38

Backlog vs Kanban Board

Two complementary tools in Agile

- Both the **backlog** and the **Kanban board** are tools for managing work, but they serve different purposes.
- The backlog defines **what should be done**, while the Kanban board shows **what is being done right now**.

	Backlog	Kanban Board
Purpose	Plan and prioritize future work	Visualize and control current work
Content	All upcoming items: stories, bugs, spikes, tasks	Only active items in the current flow
Focus	"What should we do next?"	"What are we doing now?"
Ownership	Product Owner / team	Whole team
Structure	Ordered list by priority	Columns representing workflow (To Do → In Progress → Done)
Dynamics	Updated during planning	Updated continuously during work

21/38

Tasks I

Tasks

- Each story consists of one or more tasks.
- Tasks are owned by individual team members.
- Estimated in hours (typically 4–8 hours).
- Progress is tracked via **task burndown**.
- Tasks can also support refactoring, research, or infrastructure work.



Tasks II

Example:

Story: As a registered user, I can reset my password so that I can regain access. Tasks:

- Define acceptance criteria — Anna
- Design password reset form — Marek
- Implement backend API — Ola
- Write automated tests — Ola
- Update documentation — Kasia

Spikes

What is a Spike?

A **spike** is a special type of user story used for **research, design, or exploration**. It helps the team reduce uncertainty or risk before implementation.

Characteristics:

- ▶ Time-boxed (typically 1–2 days).
- ▶ Produces knowledge, not code — outcome may be a prototype, analysis, or decision.
- ▶ Helps clarify estimates, technical feasibility, or architectural approach.
- ▶ Tracked in the backlog like other stories, but marked as *spike*.

Example:

Spike: Evaluate database options for scalable user session storage. Goal: compare PostgreSQL, Redis, and DynamoDB based on latency and cost.

9. Databases

Role in software engineering

- Data outlives applications.
- Data model quality directly affects maintainability, performance, scalability.

Data modeling flow

1. **Domain model**
2. **Logical model** (ER)
3. **Physical model** (SQL / NoSQL)

ER modeling

- **Entities, attributes, relationships.**
- Cardinalities: 1:1, 1:N, M:N.
- M:N requires **associative entity**.
- **Primary keys** (prefer surrogate keys).
- **Foreign keys** enforce integrity.

Normalization

- **1NF**: atomic attributes
- **2NF**: no partial dependency
- **3NF**: no transitive dependency
- Start normalized, denormalize only for proven performance needs.

SQL vs NoSQL

- **SQL**: ACID, strong consistency, complex queries.
- **NoSQL**: scalability, flexible schema, specialized models.
- Types: Document, Key-Value, Column-Family, Graph.
- **Polyglot persistence** when justified.

Integration

- Repository pattern
- ORM pros/cons
- N+1 problem and solutions

- Schema migrations (Flyway, Liquibase, etc.)

Anti-patterns

- God tables
- Missing foreign keys
- Shared database in microservices
- Premature NoSQL choice

Source: SE_09_10.pdf, pp. 4–46

10. Microservices Architecture

Definition

- Architectural style where an application is built as **independently deployable services**.
- Each service owns its **data and domain logic**.

Key characteristics

- Small, business-focused services.
- Independent deployment and scaling.
- Technology heterogeneity.
- Fault isolation.

Communication

- Synchronous: REST, gRPC.
- Asynchronous: messaging, events.

Supporting components

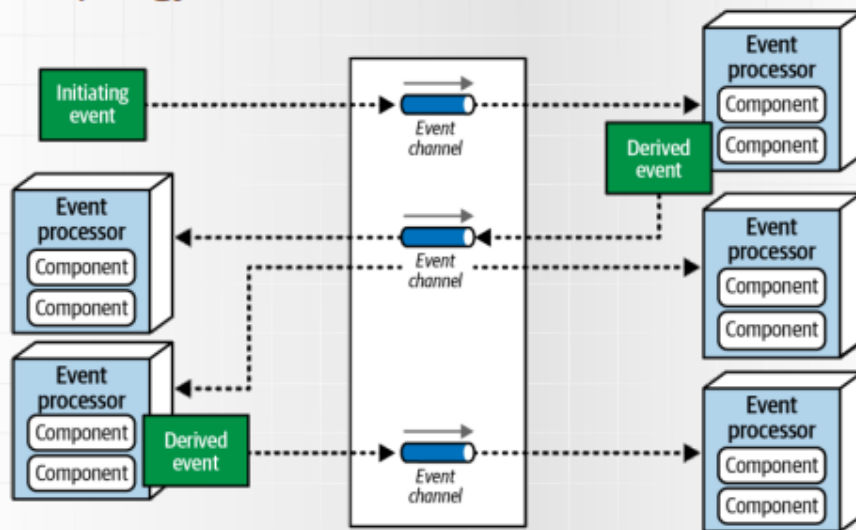
- API Gateway.
- Service discovery/registry.
- Centralized configuration.

Trade-offs

- **Pros:** scalability, agility, resilience.
- **Cons:** operational complexity, data consistency, testing difficulty.

Source: SE_05.pdf, pp. 29–31

EDA Topology



Architecture Trade-offs

- ▶ **Advantages:** Loose coupling, scalability, real-time responsiveness
- ▶ **Disadvantages:** Eventual consistency, debugging complexity, event ordering challenges
- ▶ **Best for:** Systems requiring high responsiveness and decoupling

29/40

Microservices architecture

Microservices Architecture

- ▶ An architectural style that structures an application as a collection of **loosely coupled services**.
- ▶ Each service is **independently deployable** and **scalable**.
- ▶ Services use **lightweight protocols** (HTTP/REST, messaging).
- ▶ Each service owns its own **data and domain logic**.
- ▶ Enables **technology diversity** and **independent evolution**.

Service Characteristics

- ▶ **Small and focused:** Each service addresses a specific business capability
- ▶ **Independent:** Services can be deployed without affecting others
- ▶ **Technology agnostic:** Different services can use different technologies
- ▶ **Resilient:** Failure in one service doesn't bring down the entire system
- ▶ **Scalable:** Individual services can be scaled independently

Key Components

- ▶ **API Gateway:** Single entry point for client requests
- ▶ **Service Registry/Discovery:** Dynamic service location and registration
- ▶ **Configuration Management:** Centralized configuration for all services

30/40

Microservices: Communication & Trade-offs

Communication

- **Synchronous:** Direct request-response (REST, gRPC)
- **Asynchronous:** Message queues, event streaming for decoupling
- Services can use different protocols based on needs

Data Management

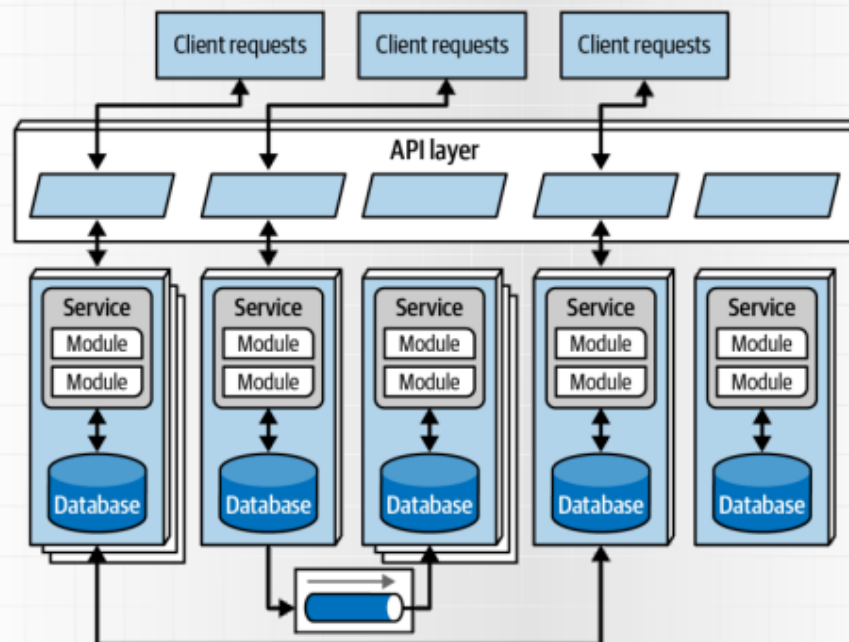
- **Database per service:** Each service owns its data store
- **Eventual consistency:** Data consistency across services handled via events
- Distributed data requires careful design and coordination

Architecture Characteristics

- **Strengths:** Scalability, technology flexibility, independent deployment, fault isolation
- **Weaknesses:** Operational complexity, distributed system challenges, testing difficulty, data consistency
- **Trade-off:** Agility and scalability vs. complexity and operational overhead

31/40

Microservices architecture



32/40

Images from the exam:

Software Engineering
Zero-term exam

Variant: 20

First name: Date:

Student ID:

1. Which statement best describes the **«extends»** relationship in a use case diagram?
 - ☐ A) It is used to factor out common behavior that is always executed.
 - ☐ B) It represents inheritance between use cases.
 - ☐ C) It adds optional behavior that is executed conditionally.
 - ☐ D) It denotes a communication link between an actor and a use case.
2. In UML *multiplicity*, what is the precise semantics of the *asterisk* symbol (*)?
 - ☐ A) It indicates that the upper bound of the association end is not specified.
 - ☐ B) It represents a mathematical infinity of instances allowed by the system.
 - ☐ C) It acts as a wildcard, meaning either 0 or 1 instance is required.
 - ☐ D) It is a reserved symbol for elements that must be dynamically allocated or stored as pointers.
3. A developer models a conditional branch using an **alt** fragment with two operands: the first has a guard $[x > 0]$ and the second operand is left completely empty. Which of the following is true?
 - ☐ A) The model is semantically equivalent to an **opt** fragment with the guard $[x > 0]$.
 - ☐ B) The model is syntactically invalid because every **alt** operand must contain at least one message.
 - ☐ C) This notation is used to represent a **critical** region that blocks other lifelines.
 - ☐ D) If $x \leq 0$, the diagram execution hangs because the "else" path has no defined behavior.
4. A **User** class contains methods for: **getName()**, **calculatePay()**, and **saveToDatabase()**. According to the Principle **S** from SOLID, why is this design flawed?
 - ☐ A) Because it depends on abstractions instead of concrete objects.
 - ☐ B) Because it is closed for extension but open for modification.
 - ☐ C) Because the class has multiple reasons to change (HR logic change vs. Database schema change).
 - ☐ D) Because it uses too many client-specific interfaces.
5. In the C4 model, what is the primary technical distinction between a **Container** and a **Component**?
 - ☐ A) A Container is used only for third-party external services, while Components are used for internal code.
 - ☐ B) A Container represents a UML Class, while a Component represents a UML Package.
 - ☐ C) A Container is a separately deployable or runnable unit (e.g., a database or web app), whereas a Component is a logical grouping of logic within a Container.
 - ☐ D) A Container represents a physical server, while a Component represents a software package.
6. What is the main goal of the *Ubiquitous Language* in Domain-Driven Design?
 - ☐ A) To force all departments in a company to use exactly the same database schema.

1

- ☐ B) To ensure that developers use only technical jargon that business experts don't understand.
 - ☐ C) To translate all domain terms into a standardized set of UML Diagrams.
 - ☐ D) To create a shared language between developers and domain experts that is used in both conversation and code.
7. Both Adapter and Decorator patterns involve wrapping an object. What is the primary difference in their intent?
- ☐ A) Adapter changes the object's behavior, while Decorator only changes its interface.
 - ☐ B) There is no difference; the terms are used interchangeably in modern software architecture.
 - ☐ C) Decorator adds responsibilities to an object dynamically, while Adapter converts an incompatible interface into one the client expects.
 - ☐ D) Adapter is used to implement the Singleton pattern, whereas Decorator is used for Strategy.
8. In an Agile environment, what is the fundamental difference between a User Story and a Task?
- ☐ A) User Stories are tracked on the Kanban board, while Tasks are only kept in the Product Backlog.
 - ☐ B) User Stories are owned by individual members, while Tasks are owned by the Product Owner.
 - ☐ C) User Stories represent business value for the user, while Tasks are the technical steps required to implement them, often estimated in hours.
 - ☐ D) A single Task can consist of multiple User Stories, whereas a Story is always a single unit of work.
9. In which of the following scenarios is a Relational Database the most appropriate choice over a NoSQL solution?
- ☐ A) When the system requires massive horizontal scalability for petabytes of unstructured IoT sensor data.
 - ☐ B) When the primary goal is to store JSON-like documents with deeply nested structures and no joins.
 - ☐ C) When data integrity and ACID transactions are critical, such as in a financial system or bank transfers.
 - ☐ D) When the schema is frequently changing and simple key-based lookups are the only access pattern.
10. In a microservices architecture, what is the primary reason for following the "Database per Service" pattern?
- ☐ A) To reduce the cost of database licenses by using smaller, free databases.
 - ☐ B) To ensure that services are loosely coupled and can be deployed or scaled independently without shared data constraints.
 - ☐ C) To simplify complex SQL joins by having all services access the same database tables.
 - ☐ D) To guarantee immediate ACID consistency across the entire distributed system.